

Berufsakademie Sachsen

Staatliche Studienakademie Leipzig

Cybernetics Oriented Programming (CYBOP)

Entwickeln und Debuggen

des Interpreters CYBOI mit Code::Blocks

Auftragnehmer: Tobias Donix, Kevin Klein

Auftraggeber / Projektleiter: Prof. Dr. Christian Heller
Berufsakademie Sachsen

Staatliche Studienakademie Leipzig

Schönauer Straße 113a

04207 Leipzig

Leipzig, 15.12.2016

Inhalt

1	Einleitung	3
1.1	Ziel der Projektarbeit	3
1.2	Code::Blocks Voraussetzungen und Installation	3
2	CYBOI Projektdatei.....	4
2.1	Aufbau der Projektdatei	4
2.2	Projektdatei importieren.....	4
2.3	Projektdatei beim Start laden	6
2.4	CYBOL-Programm als Startparameter einstellen.....	7
3	CYBOI nutzen mit Code::Blocks	8
3.1	CYBOL-Programme ausführen	8
3.2	Breakpoints setzen.....	8
3.3	Interpreter CYBOI debuggen.....	9
3.4	Überwachen von Variablen	10
4	Zusammenfassung und Ausblick	11
4.1	Versionsstand der Projektarbeit	11
4.2	Sinnvolle Ergänzungen.....	11
5	Verzeichnisse.....	12
5.1	Abkürzungsverzeichnis / Glossar	12
5.2	Abbildungsverzeichnis.....	13
5.3	Quellenverzeichnis	13

1 Einleitung

1.1 Ziel der Projektarbeit

Ziel dieser Projektarbeit im Studienwahlfach „Cybernetics Oriented Programming (CYBOP)“ ist es, CYBOP im Rahmen des Unterrichts entweder direkt weiterzuentwickeln und neue Beispielprogramme in der interpretierten Sprache CYBOL zu schreiben, oder einen anderweitigen Mehrwert zum Projekt beizusteuern.

Die vorliegende Arbeit beschäftigt sich mit der theoretischen und praktischen Möglichkeit, den Interpreter des Projektes (CYBOI) in einer Integrierten Entwicklungsumgebung (IDE) weiterentwickeln, bauen und debuggen zu können. Außerdem sollen CYBOL-Programme über diese IDE ausführbar sein. Um diesen nützlichen Mehrwert für das CYBOP-Projekt zu erreichen, wurde die freie IDE Code::Blocks ausgewählt und darin eine Projektdatei für CYBOI erstellt.

Diese Dokumentation gliedert sich folgendermaßen: in Abschnitt 1.2 wird zunächst kurz auf die Installation von Code::Blocks eingegangen. In Kapitel 2 werden der Aufbau der Projektdatei skizziert und der Import dieser Projektdatei sowie wichtige Einstellungen besprochen. Das Ausführen von CYBOL-Programmen und das Debuggen von CYBOI wird in Kapitel 3 besprochen. Im Anschluss fasst Kapitel 4 die wichtigsten Aspekte der Projektarbeit zusammen und gibt einen Ausblick auf eine mögliche Fortführung.

1.2 Code::Blocks Voraussetzungen und Installation

Code::Blocks ist eine freie Entwicklungsumgebung, die für eine Vielzahl von Plattformen verfügbar ist. Voraussetzung für die Verwendung im CYBOP-Projekt ist zunächst die ordnungsgemäße Installation von CYBOP unter einer Linux-Distribution. Eine gute Dokumentation dazu bietet die CYBOP-Homepage [1]. Anschließend kann Code::Blocks installiert werden. Auf der Code::Blocks-Homepage unter dem Menüpunkt *Downloads* → *Download the binary release* [3] finden sich die nötigen Dateien. Im Rahmen dieser Dokumentation wurden CYBOP und Code::Blocks unter OpenSuse 13.2 eingerichtet.

2 CYBOI Projektdatei

2.1 Aufbau der Projektdatei

Wie in der Einleitung unter Kapitel 1.1 beschrieben, wurde für den Interpreter CYBOI eine Projektdatei für Code::Blocks erstellt und speziell angepasst. Die Projektdateien von Code::Blocks sind in XML definiert. Abb. 1 zeigt den Kopf der Projektdatei CYBOI.cbp. In den verschiedenen XML-Sektionen können z.B. Angaben zum verwendeten Compiler, Build-Parameter und Verzeichnisse gesetzt werden.

```

1 <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2 <CodeBlocks_project_file>
3   <FileVersion major="1" minor="6" />
4   <Project>
5     <Option title="CYBOI" />
6     <Option makefile_is_custom="1" />
7     <Option execution_dir="../../trunk" />
8     <Option pch_mode="2" />
9     <Option compiler="clang" />
10    <Option show_notes="0" />
11    <notes>
12      <![CDATA[Code::Blocks Projektdatei für CYBOI. Autoren: Tobias Donix, Kevin Klein.]]>
13    </notes>
14  </option>
15  <Build>
16    <Target title="Debug">
17      <Option platforms="Unix;Mac;" />
18      <Option output="../../src/controller/cyboi" prefix_auto="1" extension_auto="1" />
19      <Option working_dir="../../examples/" />
20      <Option object_output="../../src/controller/" />
21      <Option type="1" />
22      <Option compiler="clang" />
23      <Option parameters="hello/run.cyboi" />
24      <MakeCommands>
25        <Build command="$make" />
26        <CompileFile command="$make -f $makefile $file" />
27        <Clean command="$make -f $makefile clean$target" />
28        <Distclean command="$make -f $makefile distclean$target" />
29        <AskRebuildNeeded command="$make -q -f $makefile $target" />
30        <SilentBuild command="$make &gt; $(CMD_NULL)" />
31      </MakeCommands>
32    </Target>
33    <Target title="Release">
34      <Option output="../../src/controller/cyboi" prefix_auto="1" extension_auto="1" />
35      <Option object_output="../../src/controller/" />
36      <Option type="1" />
37      <Option compiler="clang" />
38    </Target>
39  </Build>
40 </Project>
41 </CodeBlocks_project_file>

```

Abb. 1: CYBOI-Projektdatei für die IDE Code::Blocks

2.2 Projektdatei importieren

Um mit der IDE Code::Blocks am Interpreter CYBOI arbeiten und CYBOL-Programme ausführen zu können, muss nach der Installation der IDE zunächst die Projektdatei importiert werden. Dafür muss folgendermaßen vorgegangen werden:

- es muss zunächst sichergestellt werden, dass eine aktuelle Version von CYBOP ausgecheckt, ordnungsgemäß konfiguriert und gebaut wurde
- es muss weiterhin sichergestellt sein, dass Code::Blocks auf dem Zielsystem System korrekt installiert wurde
- es sollte außerdem der Compiler „LLVM clang“ auf dem Zielsystem installiert sein. Alternativ kann der verwendete Compiler in der Projektdatei angepasst werden (siehe dazu Kap. 2.1 und Code::Blocks Wiki [4])
- nun wählt man in Code::Blocks im Menü *File* den Menüpunkt *Open..* und navigiert im CYBOP-Verzeichnis in den Unterordner *ide/codeblocks/*
- schließlich kann die Projektdatei CYBOI.cbp ausgewählt und importiert werden

Jetzt sollte die Projektdatei ordnungsgemäß geöffnet sein. Abb. 2 zeigt das CYBOI-Projekt in Code::Blocks beispielhaft.

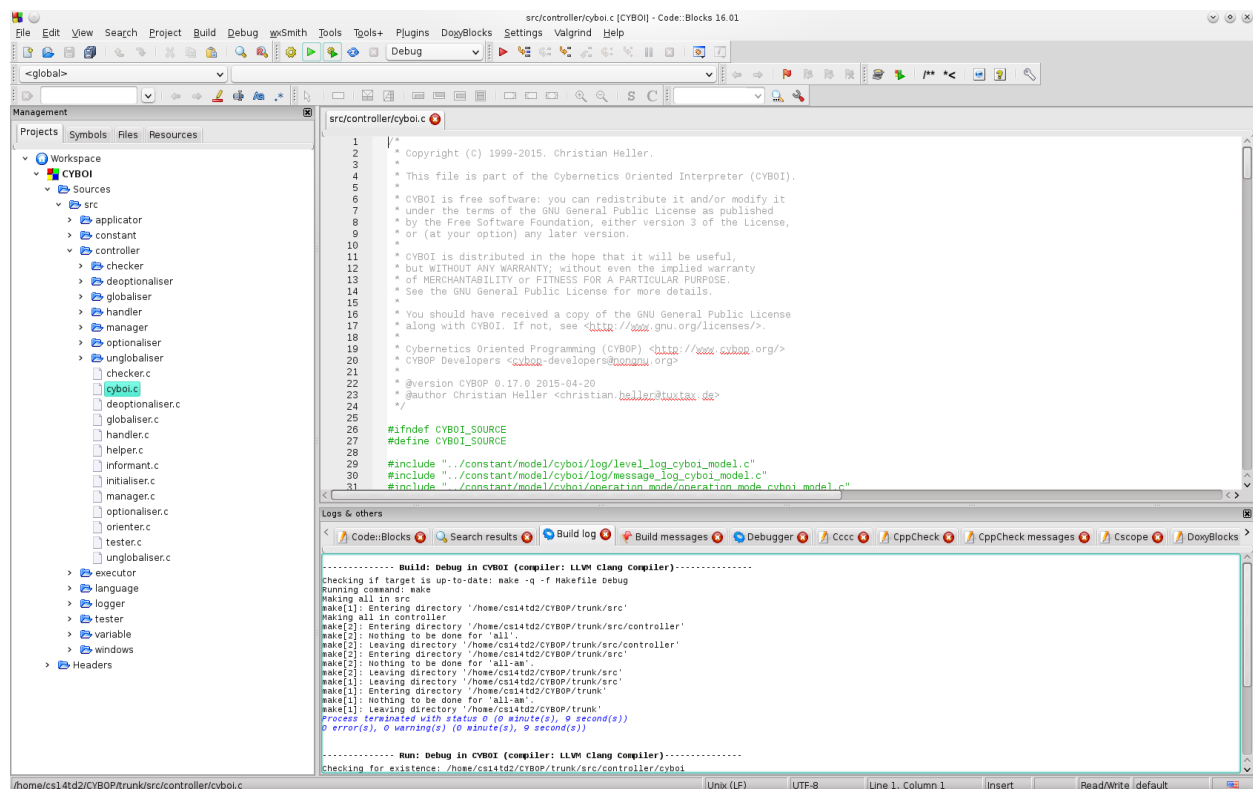


Abb. 2: CYBOI als Code::Blocks-Projekt

2.3 Projektdatei beim Start laden

Um den in Kapitel 2.2 beschriebenen Schritt nicht bei jedem Start ausführen zu müssen, empfiehlt sich folgende Einstellung:

- man öffnet im Menü *Settings* den Menüpunkt *Environment...*
- stellt im Bereich *On application start-up* die Option *Open default workspace*
- und im Bereich *On project load* die Option *Do not open any files* ein

Durch diese Einstellungen (siehe auch Abb. 3) wird das CYBOI-Projekt künftig immer mit Code::Blocks geöffnet. Die zweite Einstellung stellt zudem sicher, dass nicht bei jedem Start sämtliche zum Projekt gehörigen Dateien geöffnet werden.

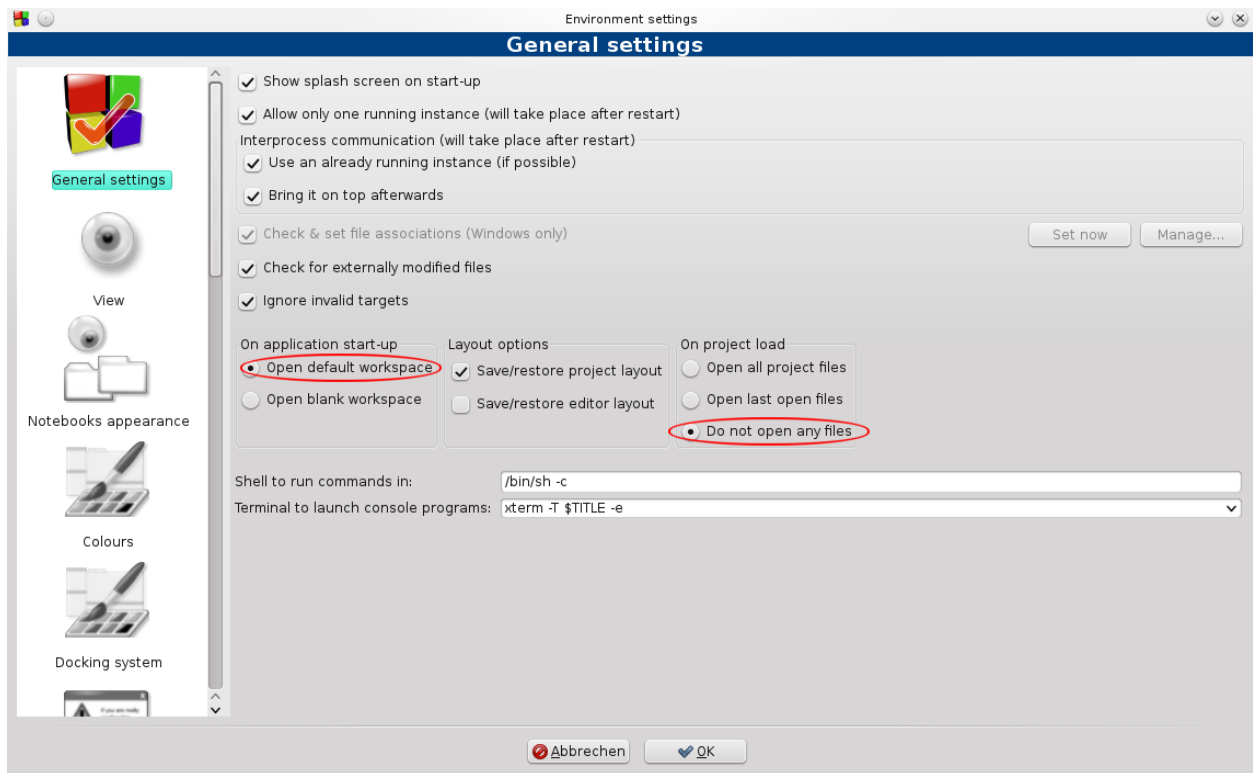


Abb. 3: Umgebungs-Einstellungen in Code::Blocks

2.4 CYBOL-Programm als Startparameter einstellen

Um ein bestimmtes CYBOL-Programm mit dem Interpreter CYBOI über Code::Blocks ausführen zu können, muss folgende Einstellung vorgenommen werden:

- im Menü *Project* öffnet man den Menüpunkt *Set programs' arguments...*
- und trägt den relativen Pfad zum gewünschten CYBOL-Programm ein (Abb. 4)

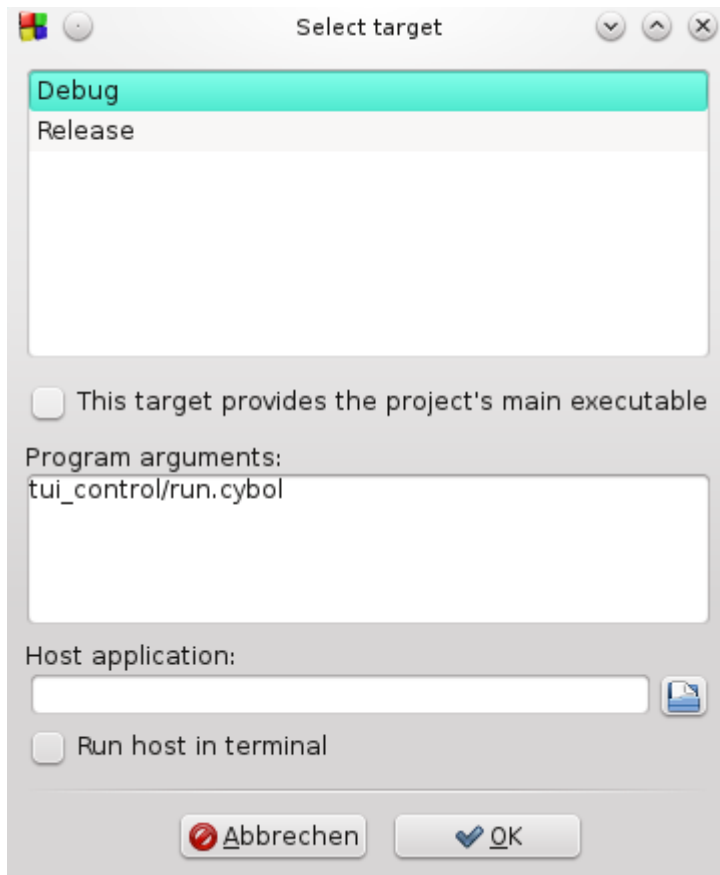


Abb. 4: Einstellen eines CYBOL-Programmes als Startparameter

Hinweise: Als Arbeitsverzeichnis ist in der Projektdatei das CYBOP-Unterverzeichnis *examples* bereits voreingestellt (`<Option working_dir="../../examples" />`). Das gewünschte Beispielprogramm wird im Bauziel (build target) „Debug“ eingetragen (vgl. auch 4.1).

- man öffnet im Menü *Debug* den Menüpunkt *Debugging windows*
- und setzt dort ein Häkchen an der Auswahl *Breakpoints*

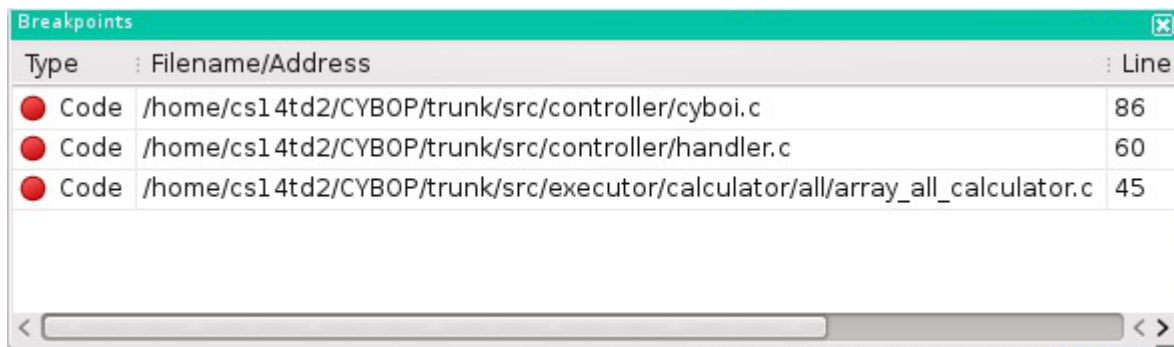


Abb. 7: Debugging-Fenster Breakpoints in Code::Blocks

3.3 Interpreter CYBOI debuggen

Um den Interpreter CYBOI mit Code::Blocks zu debuggen, gehen wie folgt vor:

- es muss zunächst sichergestellt werden, dass ein vorhandenes CYBOL-Programm als Startparameter eingestellt wurde (siehe Kap. 2.4)
- weiterhin sollten dem ausgewählten CYBOL-Programm entsprechend sinnvolle Breakpoints in CYBOL gesetzt sein (siehe Kap. 3.2)
- Nun kann mit der Taste F8 oder einem Klick auf den roten Pfeil in der Debugging-Bedienleiste (Abb. 8) das Debuggen gestartet werden



Abb. 8: Bedienleiste zum Debuggen in Code::Blocks

Nun kann man entsprechend der gesetzten Breakpoints schrittweise die Programmverarbeitung durch CYBOI debuggen. Wie in anderen IDEs auch können in Code::Blocks folgende Tasten bzw. Tastenkombination zum Debuggen verwendet werden:

- F8 *Start / Continue* (Start / nächster Breakpoint)

- F7 *Next line (nächste Zeile)*
- Umschalt + F7 *Step into (Ausführen der nächsten Funktion)*
- Strg + F7 *Step out (Hinweggehen über die nächste Funktion)*
- Alt + F7 *Next instruction (nächste Anweisung)*
- Umschalt + Alt + F7 *Step into instruction*

Die genannten Optionen können auch über Klicken auf die entsprechenden Icons in der Bedienleiste ausgeführt werden (Abb. 8 und 9). Ein laufender Debug-Vorgang kann durch Klicken auf das rote Kreuz (Abb. 9) oder drücken von Umschalt + F8 zu jeder Zeit beendet werden.

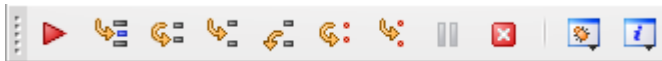


Abb. 9: Beenden eines Debug-Vorgangs in Code::Blocks

3.4 Überwachen von Variablen

Während eines laufenden Debug-Vorgangs können auch Variablen überwacht werden. Dazu geht man folgendermaßen vor:

- Klick mit der rechten Maustaste auf die zu überwachende Variable
- Auswahl der Option *Watch [variable]*

Außerdem muss noch das Debugging-Fenster *Watches* aufgerufen werden, um die überwachten Variablen und deren Inhalte sehen zu können. Dieses Fenster kann im Menü *Debug* unter dem Menüpunkt *Debugging windows* aufgerufen werden (vgl. auch Kap. 3.2).

4 Zusammenfassung und Ausblick

4.1 Versionsstand der Projektarbeit

Mit Hilfe der im Rahmen des Wahlfaches Cybernetics Oriented Programming (CYBOP) erstellten Projektdatei für die integrierte Entwicklungsumgebung Code::Blocks und der zugehörigen Dokumentation kann der Cybernetics Oriented Interpreter (CYBOI) komfortabel weiterentwickelt und debuggt werden. Die Code::Blocks-Projektdatei stellt somit eine wertvolle Hilfe für Entwickler dar. Auch das Bauen einer geänderten Version sowie das Ausführen eines beliebigen CYBOL-Programmes kann direkt über die IDE erfolgen.

Im aktuellen Projektstand wurde vor allem Wert auf die Möglichkeit des Debugging gelegt. Dementsprechend wurde das Bauziel (build target) „Release“ in der IDE zum gegenwärtigen Zeitpunkt noch nicht vollständig konfiguriert (vgl. auch Kapitel 4.2).

4.2 Sinnvolle Ergänzungen

In einer Fortführung des Projektes könnten folgende Aspekte behandelt werden:

- Konfiguration bzw. Anpassung des Bauziels „Release“
- Dokumentation nützlicher Plugins wie AutoVersioning, Code statistics und SVN Integration (siehe dazu auch Code::Blocks Wiki unter dem Menüpunkt Code::Blocks Plugins [4] oder Handbuch, Kapitel 2 [2])

5 Verzeichnisse

5.1 Abkürzungsverzeichnis / Glossar

Bauen	Der gesamte Erstellungsprozess einer Software (engl. Build), der in der Erstellung eines lauffähigen Produktes mündet. Das Produkt kann eine Testversion oder eine auszuliefernde Version sein. Der Prozess kann auch weitere Schritte beinhalten wie das Ausführen von Tests, die Erstellung einer Dokumentation oder die Erhebung von Metriken.
Bauziel	(auch build target) Ein Ziel, dass im Rahmen des Bauvorgangs angestrebt wird, z. B. die Veröffentlichung einer fertigen Version eines Programmes (Release).
Breakpoint	Ein vom Programmierer definierter Punkt bzw. eine Codezeile, an welcher der Debugger den Programmfluss anhalten soll.
CYBOI	Cybernetics Oriented Interpreter. Der Interpreter, der CYBOL-Programme übersetzt und ausführt.
CYBOL	Cybernetics Oriented Language. Eine XML-basierte, plattformunabhängige Sprache.
CYBOP	Cybernetics Oriented Programming. Ein neues Konzept für die Softwareentwicklung, welches auf Prinzipien aus der Natur aufbaut.
Distribution	Zum Beispiel in Bezug auf Linux eine bestimmte Zusammenstellung von Softwarekomponenten, die in ihrer Summe das lauffähige Betriebssystem auf Linux-Basis bildet. Traditionell sind bei Linux-Distributionen z. B. die grafischen Oberflächen variabel [5].
IDE	Integrated Development Environment (Integrierte Entwicklungsumgebung). Programm bzw. Editor, der unterstützende Zusatzfunktionen zum Schreiben von Programmen bietet. Beispiele sind Code-Hervorhebungen und -Vervollständigungen.

5.2 Abbildungsverzeichnis

Abb. 1: CYBOI-Projektdatei für die IDE Code::Blocks	4
Abb. 2: CYBOI als Code::Blocks-Projekt	5
Abb. 3: Umgebungs-Einstellungen in Code::Blocks	6
Abb. 4: Einstellen eines CYBOL-Programmes als Startparameter	7
Abb. 5: Bedienelemente zum Bauen und Ausführen in Code::Blocks	8
Abb. 6: Setzen von Breakpoints in Code::Blocks.....	8
Abb. 7: Debugging-Fenster Breakpoints in Code::Blocks.....	9
Abb. 8: Bedienleiste zum Debuggen in Code::Blocks.....	9
Abb. 9: Beenden eines Debug-Vorgangs in Code::Blocks	10

5.3 Quellenverzeichnis

- [1]: Heller, C. (2016): "Cybernetics Oriented Programming (CYBOP)" (Homepage), unter: <http://www.nongnu.org/cybop/index.html> (aufgerufen am 24.10.2016)
- [2]: The Code::Blocks Team (2010): „CodeBlocks Manual Version 1.1 (deutsch)“, unter: http://www.codeblocks.org/docs/manual_de.pdf (aufgerufen am 27.11. 2016)
- [3]: The Code::Blocks Team (2016): "Code::Blocks - The IDE with all the features you need, having a consistent look, feel and operation across platforms.", unter: <http://www.codeblocks.org> (aufgerufen am 27.11.2016)
- [4]: The Code::Blocks Team (2016): „The official Wiki for Code::Blocks“, unter: http://wiki.codeblocks.org/index.php/Main_Page (aufgerufen am 27.11.2016)
- [5]: Wikipedia, die freie Enzyklopädie (2016): "Distribution (Software)", unter: [https://de.wikipedia.org/wiki/Distribution_\(Software\)](https://de.wikipedia.org/wiki/Distribution_(Software)) (aufgerufen am 27.11.2016)